

architecture

DocProcessor Architecture

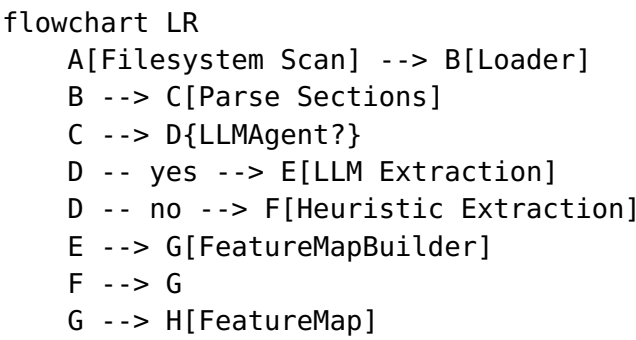
Module: `digital.vasic.docprocessor`

DocProcessor loads project documentation, extracts structured feature maps, tracks verification coverage, and builds inter-document link graphs. It is designed for injection into QA pipelines (e.g., HelixQA autonomous sessions) but also runs standalone via `cmd/docprocessor`.

Package Overview

Package	Role
pkg/loader	Document discovery and parsing
pkg/feature	Feature extraction and FeatureMap construction
pkg/coverage	Thread-safe verification coverage tracking
pkg/docgraph	Inter-document link graph with export
pkg/llm	Injected LLM interface for intelligent extraction
pkg/config	Configuration from <code>.env</code> files

Processing Pipeline



```
H --> I[CoverageTracker]
H --> J[DocGraph]
```

1. **Scan** — `loader.Scanner` walks the project tree respecting `HELIX_DOCS_FORMATS` (md, yaml, html, adoc, rst) and the `HELIX_DOCS_ROOT` setting.
 2. **Load & Parse** — `loader.Loader` reads each file (max 10 MB), splits it into `Section` structs with headings, bodies, and cross-reference links.
 3. **Extract Features** — `feature.FeatureMapBuilder` runs heuristic extraction by default. When an `llm.LLMAgent` is injected it sends prompt templates to the agent and parses `RawFeature` responses.
 4. **Build FeatureMap** — Extracted features are deduplicated (deterministic IDs via `GenerateID`), categorised, and assembled into a queryable `FeatureMap` with a platform matrix.
 5. **Enrich** — The optional LLM pass infers UI screens and generates test step suggestions for each feature.
 6. **Track Coverage** — `coverage.CoverageTracker` records per-platform verification status, stores `Evidence`, and produces `CoverageReport` snapshots.
-

Loader Pipeline

`loader.Loader` accepts a root path and format list. Internally it chains:

- **MarkdownParser** — extracts ATX/setext headings and fenced code blocks.
- **YAMLParse** — unmarshals structured docs via `gopkg.in/yaml.v3`.
- **HTMLParser** — strips tags and extracts heading hierarchy.
- **AsciiDocParser / RSTParser** — plain-text heading extraction.

All parsers return `[]loader.Document` with `[]Section` slices.

Feature Extraction

`feature.FeatureMapBuilder` operates in two modes:

- **Heuristic** — Keyword scoring against section headings and bodies to assign `FeatureCategory` (functional, UI, API, configuration, etc.) and detect platform tags (android, web, desktop).
- **LLM-powered** — Sends a structured prompt to the injected `llm.LLMAgent` and deserialises the returned `[]RawFeature` slice into `Feature` structs.

Feature IDs are deterministic: short names become slugs; long names are slug + 6-char SHA256 suffix, ensuring stable references across pipeline runs.

Coverage Tracking

`coverage.CoverageTracker` is fully thread-safe:

- Read paths (`GetReport`, `GetStatus`) use `sync.RWMutex.RLock()`.
- Write paths (`MarkVerified`, `RecordIssue`, `AttachEvidence`) use `Lock()`.

A `CoverageReport` snapshot captures per-feature, per-platform status at a point in time and is serialisable to JSON for downstream consumers (e.g., HelixQA reporter).

DocGraph

`pkg/docgraph.DocGraph` maintains a directed graph of document nodes and typed edges (reference, include, related). It is protected by `sync.RWMutex` and can export to:

- **JSON** — full node/edge list for programmatic consumers.
 - **Mermaid** — flowchart LR diagram for documentation embedding.
-

LLMAgent Interface

```
type LLMAgent interface {  
    Extract(ctx context.Context, prompt string) ([]RawFeature,  
        error)  
}
```

The interface carries no module-level dependency on `LLMOrchestrator` or any concrete provider. Callers inject a conforming implementation at construction time, keeping `DocProcessor` buildable and testable without any LLM credentials.